Quero que você atue como arquiteto e desenvolvedor sênior em **Java Web**, ajudando a projetar e implementar um sistema para geração de arquivos XML de **Ressarcimento de ICMS-ST** para a SEFAZ/AM, seguindo rigorosamente o "Manual de Orientação do Contribuinte – Ressarcimento de ICMS Substituição Tributária – SEFAZ/AM – 19/03/2019" e o fluxo real de envio via DT-e (Domicílio Tributário Eletrônico).

---

1. Contexto funcional e regras de negócio

Use como base obrigatória o seguinte conteúdo extraído do manual oficial:

1.1 Arquivo de Produtos – MATRI-NAC

- O contribuinte obrigado ao pagamento do ICMS Antecipado por Declaração (art. 9º do Decreto 32.128/2012 e Resolução 0008/2012) deve apresentar um **arquivo inicial** contendo os produtos adquiridos de outras UFs, destinados à comercialização, industrialização, prestação de serviço, ativo permanente ou uso e consumo.
- Esse arquivo é denominado **Matriz Nacional (MATRI-NAC)** e será auditado e confrontado com os arquivos digitais enviados mensalmente à SEFAZ.
- Segue o padrão **XML**, schema: `enviProdutoRessarcimento_v9.99.xsd`.

Leiaute do Arquivo de Produtos (conforme manual, seção 1.1):

- Tag raiz: `enviProdutoRessarcimento` (A01)
- `versao` (B01) – Tipo N, Ocorr 1-1, Tam 1-4, Dec 2

Bloco `dadosDeclarante` (C01) – Tipo G, Ocorr 1-1:

- `cnpjRaiz` (C02) – Tipo N, Ocorr 1-1, Tam 8
- `razaoSocial` (C03) – Tipo C, Ocorr 1-1, Tam 3-60
- `nomeResponsavel` (C04) – Tipo C, Ocorr 1-1, Tam 3-60
- `foneResponsavel` (C05) – Tipo C, Ocorr 1-1, Tam 7-15
- `emailResponsavel` (C06) – Tipo C, Ocorr 1-1, Tam 3-60

Bloco `listaProdutos` (D01) – Tipo G, Ocorr 1-1, contendo N ocorrências de `produto` (D02):

- `codInternoProduto` (D03) – Tipo C, Ocorr 1-1, Tam 1-60
- `descricaoProduto` (D04) – Tipo C, Ocorr 1-1, Tam 1-100
- `unidadeInternaProduto` (D05) – Tipo C, Ocorr 1-1, Tam 2-8

- `fatorConversao` (D06) – Tipo N, Ocorr 1-1, Tam 1-9, Dec 6 (quantidade de itens que alimentam o estoque do contribuinte para cada unidade comprada do fornecedor)
- `cnpjFornecedor` (D07) – Tipo N, Ocorr 1-1, Tam 14
- `codProdFornecedor` (D08) – Tipo C, Ocorr 1-1, Tam 1-60
- `unidadeProdutoFornecedor` (D09) – Tipo C, Ocorr 1-1, Tam 2-8

1.2 Arquivo de Pedidos – Versão 2.00 (todas as modalidades exceto PMPF)

Importante: neste projeto **não implementar** o leiaute PMPF (Versão 1.00). Apenas o **arquivo de pedidos Versão 2.00**. O suporte ao PMPF poderá ser adicionado futuramente como extensão planejada.

Schema XML: `enviOperacaoRessarcimento_v9.99.xsd`

Leiaute do Arquivo de Pedidos – Versão 2.00 (conforme manual, seção 2.1):

- Tag raiz: `enviOperacaoRessarcimento` (A01)
- `versao` (B01) – Tipo N, Ocorr 1-1, Tam 1-4, Dec 2 → valor "2.00"

Bloco `dadosDeclarante` (C01) – Tipo G, Ocorr 1-1:

- `cnpjRaiz` (C02) – Tipo N, Ocorr 1-1, Tam 8
- `ieContribuinteDeclarante` (C03) – Tipo N, Ocorr 1-1, Tam 8-15
- `razaoSocial` (C04) – Tipo C, Ocorr 1-1, Tam 3-60
- `nomeResponsavel` (C05) – Tipo C, Ocorr 1-1, Tam 3-60
- `foneResponsavel` (C06) – Tipo C, Ocorr 1-1, Tam 7-15
- `emailResponsavel` (C07) – Tipo C, Ocorr 1-1, Tam 3-60
- `anoPeriodoReferencia` (C08) – Tipo C, Ocorr 1-1, Tam 4 (ano da operação de saída, formato AAAA)
- `mesPeriodoReferencia` (C09) – Tipo C, Ocorr 1-1, Tam 2 (mês da operação de saída, formato MM)

Bloco `listaOperacoes` (D01) – Tipo G, Ocorr 1-1, contendo N ocorrências de `operacao` (D02):

- `chaveNFe` (D03) – Tipo N, Ocorr 1-1, Tam 44 (chave da NF-e de saída)

Dentro de cada `operacao`, bloco `listaItens` (E01) – Tipo G, Ocorr 1-1, contendo N ocorrências de `item` (E02):

- `codInternoProduto` (E03) – Tipo C, Ocorr 1-1, Tam 1-60
- `numItemNFe` (E04) – Tipo N, Ocorr 1-1, Tam 1-3

- `chaveNFeEntrada` (E05) – Tipo N, Ocorr 0-1, Tam 44 (campo não obrigatório)

Bloco `listaNFeEntrada` (F01) – Tipo G, Ocorr N, contendo N ocorrências de `nfeEntrada` (F02):

- `chaveNFeEntrada` (F03) – Tipo N, Ocorr 1-1, Tam 44
- `chaveCTeEntrada` (F04) – Tipo N, Ocorr 0-1, Tam 44 (campo não obrigatório)
- `chaveMDFeEntrada` (F05) – Tipo N, Ocorr 0-1, Tam 44 (campo não obrigatório)

Observação do manual (alteração de 05/07/2022): o campo C01 foi renomeado de `dadosDeclaracao` para `dadosDeclarante`; ambos os nomes são aceitos pela SEFAZ.

1.3 Envio dos arquivos XML via DT-e

Conforme manual (seção 3) e tela do DT-e fornecida:

Arquivo de Produtos:

1. Acessar o DT-e e pesquisar "Pedido Eletrônico de Ressarcimento do ICMS-ST".
2. Selecionar "Upload Arquivo de Produtos".
3. Fazer o upload do XML e clicar em Enviar.

Arquivo de Pedido:

- Pré-requisito obrigatório: já ter enviado um Arquivo de Produtos.

1. Selecionar "Novo Pedido Eletrônico de Ressarcimento".
2. Preencher o formulário e incluir o XML do arquivo de pedido.
3. Clicar em "Cadastrar Processo" → abre caixa de assinatura digital para anexar documentos comprobatórios.
4. Após assinar todos os documentos, o pedido é finalizado.

Consulta:

- Selecionar "Listar Pedidos / Produtos".
- Aba "Produtos": todos os Arquivos de Produtos enviados.
- Aba "Pedidos": todos os pedidos enviados, com menu de opções à direita conforme situação do pedido.

---

2. Objetivo do sistema

Desenvolver um **sistema Java Web** para uso interno da empresa (ramo: **comércio**, Amazonas) que:

1. Receba **planilhas** (Excel/CSV) como fonte de dados para:
   - Cadastro de produtos (MATRI-NAC).
   - Dados do declarante (CNPJ raiz, IE, razão social, responsável, telefone, e-mail).
   - Operações de saída (NF-es de saída, itens e vinculação às NF-es de entrada, CT-e, MDF-e).
2. Valide os dados de acordo com o manual (tamanhos, formatos, obrigatoriedade, ocorrência de tags).
3. Gere automaticamente os XMLs:
   - `enviProdutoRessarcimento` (Arquivo de Produtos / MATRI-NAC).
   - `enviOperacaoRessarcimento` Versão **2.00** (Arquivo de Pedidos).
4. Permita ao usuário baixar esses XMLs para posterior upload manual no DT-e.
5. Mantenha histórico de arquivos gerados (produtos e pedidos), com logs de validação e erros.
6. **Não é necessário** acessar os XMLs das NF-e/CT-e/MDF-e originais. O sistema depende exclusivamente dos dados fornecidos nas planilhas. O preenchimento correto das chaves e itens é responsabilidade do operador que alimenta as planilhas.

---

3. Tecnologias e requisitos técnicos

3.1 Banco de dados

- **SGBD: Microsoft SQL Server**
- **Nome do banco de dados: `ressarcimento`**
- Utilizar **Spring Data JPA** com driver JDBC para SQL Server (`mssql-jdbc`).
- Configurar `application.yml` com os parâmetros de conexão:

```
  spring:
    datasource:
      url:
jdbc:sqlserver://localhost;databaseName=ressarcimento;encrypt=false
      driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
      username: <usuario>
      password: <senha>
    jpa:
      database-platform: org.hibernate.dialect.SQLServerDialect
```

```
    hibernate:
      ddl-auto: validate
```

- Utilizar **Flyway** para versionamento e criação das tabelas.
- Scripts de migração Flyway escritos em **T-SQL** (sintaxe SQL Server).
- Atenção às particularidades do SQL Server:
  - `NVARCHAR` para texto unicode.
  - `DECIMAL(15,6)` para `fatorConversao`.
  - `CHAR(44)` para chaves NF-e, CT-e, MDF-e.
  - `CHAR(8)` para `cnpjRaiz`.
  - `CHAR(14)` para `cnpjFornecedor`.
  - `DATETIME2` para timestamps de auditoria.
  - `IDENTITY(1,1)` para chaves primárias autoincrementais.
  - Evitar palavras reservadas do SQL Server como nomes de colunas (ex.: `year`, `month`, `key`, `value`).

3.2 Back-end

- Java 17+
- Spring Boot
- Spring Web / Spring MVC
- Spring Data JPA com `mssql-jdbc`
- Bean Validation (Jakarta Validation) para validar entidades e DTOs conforme regras do manual
- Leitura de planilhas com **Apache POI** (Excel .xlsx) e suporte a CSV
- Geração de XML com **JAXB**, preferencialmente com suporte a validação contra schema XSD

3.3 Front-end

- **O front-end será desenvolvido em uma fase posterior**, após a conclusão e validação completa do back-end.
- Na fase de back-end, as funcionalidades serão expostas como **APIs REST** e testadas via ferramentas como Postman, Insomnia ou testes automatizados.
- Na escolha do front-end (a ser feita na fase 2), avaliar e justificar entre:
  - **Spring MVC + Thymeleaf** (server-side, mais simples para perfis fiscais/contábeis)
  - **SPA** (React/Angular/Vue) consumindo as APIs REST já construídas
- Desde o início, projetar os controllers REST de forma limpa, independente de tecnologia de front-end, para facilitar a integração futura.

3.4 Infraestrutura e boas práticas

- Arquitetura em camadas: Controller → Service → Repository → Domain/Entity → DTOs.
- Tratamento centralizado de exceções e erros de validação (usando `@ControllerAdvice` / `@ExceptionHandler`).
- Testes unitários e de integração (JUnit 5 + Mockito; Testcontainers com SQL Server se possível).
- Versionamento de banco com **Flyway** (scripts T-SQL).
- Logs estruturados e trilha de auditoria das gerações de arquivos.

---

4. Estratégia de desenvolvimento — Backend primeiro, Frontend depois

Esta é uma diretriz central do projeto:

Fase 1 — Backend completo e validado

Desenvolver e validar integralmente o back-end antes de iniciar qualquer trabalho de front-end. O back-end deve estar:

- Com todas as regras de negócio implementadas e testadas.
- Com todas as validações conforme o manual da SEFAZ/AM funcionando corretamente.
- Com os dois XMLs sendo gerados corretamente e validados manualmente (upload no DT-e de teste ou conferência visual).
- Com APIs REST documentadas e testáveis via Postman/Insomnia.
- Com cobertura de testes unitários satisfatória nos serviços críticos (importação de planilha, geração de XML, validações).

Somente após essa validação completa do back-end, passar para a Fase 2.

Fase 2 — Frontend

- Desenvolver a interface de usuário (Thymeleaf ou SPA, a ser decidido) consumindo as APIs REST já construídas e validadas.
- A escolha da tecnologia de front-end será feita ao final da Fase 1, com base no que for mais adequado ao perfil dos usuários (fiscal/contábil) e à complexidade do projeto.

---

5. O que você deve entregar

Organize a resposta nas seguintes seções, detalhadas e com foco em boas práticas:

5.1 Arquitetura e módulos do sistema

Defina a arquitetura geral e as camadas/módulos do back-end:

- **Módulo Declarante**
  - CRUD dos dados do declarante (cnpjRaiz, IE, razão social, responsável, telefone, e-mail).
  - Alimenta o bloco `dadosDeclarante` de ambos os XMLs.
- **Módulo Produtos (MATRI-NAC)**
  - Importação de planilha de produtos (Apache POI).
  - Validação dos campos D03–D09.
  - Persistência no banco `ressarcimento`.
  - Geração do XML `enviProdutoRessarcimento`.
  - Histórico de arquivos de produtos gerados.
- **Módulo Pedidos (Versão 2.00)**
  - Importação de planilha de operações de saída.
  - Validação dos campos C02–F05.
  - Agrupamento por `chaveNFe` de saída → `listaOperacoes` / `listaItens`.
  - Deduplicação de chaves de NF-e de entrada → `listaNFeEntrada`.
  - Geração do XML `enviOperacaoRessarcimento` Versão "2.00".
  - Histórico de arquivos de pedidos gerados.
- **Módulo Importação de Planilhas**
  - Upload de arquivos Excel (.xlsx) e/ou CSV.
  - Mapeamento de colunas para entidades/DTOs.
  - Relatório de erros por linha (tipo incorreto, tamanho fora do limite, campo obrigatório ausente, chave com tamanho incorreto, `codInternoProduto` inexistente no cadastro).
- **Módulo Geração e Download de XML**
  - Serialização dos objetos de domínio para XML (JAXB).
  - Validação final contra o XSD (se aplicável).
  - Endpoint REST para download do XML gerado.

5.2 Modelo de dados — entidades e relacionamentos

Defina as principais entidades e relacionamentos, com atenção às particularidades do SQL Server:

- `Declarante`
- `ProdutoMatriz` (com dados do fornecedor — avaliar se `Fornecedor` deve ser entidade separada)
- `ArquivoProdutos` (histórico: versão, data geração, status, arquivo XML gerado)
- `NotaSaida` e `ItemNotaSaida`
- `NotaEntrada`
- `ArquivoPedido` (histórico: ano/mês de referência, versão, status, arquivo XML gerado)

Especifique:

- Quais entidades são persistidas e quais são apenas DTOs de transporte.
- Como representar o relacionamento entre `ItemNotaSaida` e `NotaEntrada`.
- Scripts Flyway em T-SQL para criação das tabelas no banco `ressarcimento`.

5.3 Modelo das planilhas de entrada

Planilha Declarante (uma linha):

Coluna na planilha	Campo XML	Tipo	Tam.	Obrigatório
cnpj_raiz	cnpjRaiz (C02)	Numérico	8	Sim
ie_contribuinte	ieContribuinteDeclarante (C03)	Numérico	8-15	Sim
razao_social	razaoSocial (C03/C04)	Texto	3-60	Sim
nome_responsavel	nomeResponsavel (C04/C05)	Texto	3-60	Sim
fone_responsavel	foneResponsavel (C05/C06)	Texto	7-15	Sim
email_responsavel	emailResponsavel (C06/C07)	Texto	3-60	Sim

Planilha Produtos (uma linha por combinação produto × fornecedor):

Coluna na planilha	Campo XML	Tipo	Tam.	Obrigatório
cod_interno_produto	codInternoProduto (D03)	Texto	1-60	Sim
descricao_produto	descricaoProduto (D04)	Texto	1-100	Sim
unidade_interna	unidadeInternaProduto (D05)	Texto	2-8	Sim
fator_conversao	fatorConversao (D06)	Decimal	1-9 / 6 dec.	Sim
cnpj_fornecedor	cnpjFornecedor (D07)	Numérico	14	Sim
cod_prod_fornecedor	codProdFornecedor (D08)	Texto	1-60	Sim
unidade_fornecedor	unidadeProdutoFornecedor (D09)	Texto	2-8	Sim

Planilha Operações de Saída (uma linha por item de NF-e de saída):

Coluna na planilha	Campo XML	Tipo	Tam.	Obrigatório
ano_referencia	anoPeriodoReferencia (C08)	Texto	4	Sim
mes_referencia	mesPeriodoReferencia (C09)	Texto	2	Sim
chave_nfe_saida	chaveNFe (D03)	Numérico	44	Sim
num_item_nfe	numItemNFe (E04)	Numérico	1-3	Sim
cod_interno_produto	codInternoProduto (E03)	Texto	1-60	Sim
chave_nfe_entrada	chaveNFeEntrada (E05/F03)	Numérico	44	Não

Coluna na planilha	Campo XML	Tipo	Tam.	Obrigatório
chave_cte_entrada	chaveCTeEntrada (F04)	Numérico	44	Não
chave_mdfe_entrada	chaveMDFeEntrada (F05)	Numérico	44	Não

Exportar

Copiar

Observações importantes sobre a Planilha de Operações:

- `ano_referencia` e `mes_referencia` são derivados da **data de emissão da NF-e de saída** (data da operação de saída que gera o direito ao ressarcimento). O operador preenche com o ano e mês da emissão da NF-e de saída correspondente.
- O sistema agrupa as linhas por `chave_nfe_saida` para montar cada bloco `<operacao>` com sua `<listaItens>`.
- O sistema coleta as chaves únicas de `chave_nfe_entrada` para montar `<listaNFeEntrada>`, sem duplicatas.
- Campos opcionais vazios na planilha → tag omitida no XML (nunca tag vazia).
- Todas as linhas de uma mesma planilha de operações devem ter o mesmo `ano_referencia` e `mes_referencia` (um arquivo de pedido = um período de referência).

5.4 APIs REST do back-end

Defina os endpoints REST principais que serão desenvolvidos e testados na Fase 1 (antes do front-end), por exemplo:

- `POST /api/declarante` – cadastrar/atualizar dados do declarante
- `GET /api/declarante` – consultar dados do declarante
- `POST /api/produtos/importar` – upload da planilha de produtos (multipart/form-data)
- `GET /api/produtos` – listar produtos cadastrados
- `POST /api/produtos/gerar-xml` – gerar o XML `enviProdutoRessarcimento` e retornar para download
- `GET /api/produtos/historico` – listar histórico de arquivos de produtos gerados
- `POST /api/pedidos/importar` – upload da planilha de operações de saída (multipart/form-data)
- `GET /api/pedidos` – listar operações importadas

- `POST /api/pedidos/gerar-xml` – gerar o XML `enviOperacaoRessarcimento` versão "2.00" e retornar para download
- `GET /api/pedidos/historico` – listar histórico de arquivos de pedidos gerados

Para cada endpoint, especifique: método HTTP, path, body/parâmetros esperados, formato de resposta (JSON ou arquivo para download) e possíveis códigos de erro.

5.5 Fluxo de uso do sistema (back-end — Fase 1)

Descreva o fluxo de uso do sistema durante a Fase 1 (sem front-end), para validação via Postman/Insomnia:

1. Cadastrar os dados do declarante via `POST /api/declarante`.
2. Fazer upload da planilha de produtos via `POST /api/produtos/importar` → sistema valida e persiste no banco `ressarcimento`.
3. Gerar o XML de produtos via `POST /api/produtos/gerar-xml` → baixar o arquivo XML.
4. Validar manualmente o XML (conferência visual + upload no DT-e em "Upload Arquivo de Produtos").
5. Fazer upload da planilha de operações via `POST /api/pedidos/importar` → sistema valida e persiste.
6. Gerar o XML de pedidos via `POST /api/pedidos/gerar-xml` → baixar o arquivo XML.
7. Validar manualmente o XML (conferência visual + upload no DT-e em "Novo Pedido Eletrônico de Ressarcimento").
8. Consultar histórico via `GET /api/produtos/historico` e `GET /api/pedidos/historico`.

5.6 Validações essenciais

Liste e detalhe todas as validações obrigatórias:

- `cnpjRaiz`: exatamente 8 dígitos numéricos.
- `cnpjFornecedor`: exatamente 14 dígitos numéricos.
- Chaves NF-e, CT-e, MDF-e: exatamente 44 dígitos numéricos.
- `fatorConversao`: numérico, máximo 9 dígitos inteiros, exatamente 6 casas decimais.
- `razaoSocial`, `nomeResponsavel`: mínimo 3, máximo 60 caracteres.
- `foneResponsavel`: mínimo 7, máximo 15 caracteres.
- `emailResponsavel`: mínimo 3, máximo 60 caracteres, formato de e-mail válido.

- `unidadeInternaProduto`, `unidadeProdutoFornecedor`: mínimo 2, máximo 8 caracteres.
- `anoPeriodoReferencia`: exatamente 4 dígitos, ano válido.
- `mesPeriodoReferencia`: exatamente 2 dígitos, valor entre 01 e 12.
- `codInternoProduto` nos itens de pedido: deve existir no cadastro de produtos persistido no banco.
- `listaNFeEntrada`: chaves não podem se repetir — deduplicação automática pelo sistema.
- Campos opcionais (E05, F04, F05): omitir a tag inteira quando não informados — nunca gerar tag vazia no XML.
- Todas as linhas de uma planilha de operações devem pertencer ao mesmo período (mesmo `ano_referencia` e `mes_referencia`).
- Relatório de erros por linha da planilha: número da linha, nome do campo, valor informado e descrição do problema.

5.7 Etapas de desenvolvimento — Roadmap

FASE 1 — Backend (desenvolver e validar completamente antes de avançar)

Etapa 1.1 – Análise e modelagem

- Refinar modelo de dados e entidades.
- Definir contratos das APIs REST.
- Criar banco `ressarcimento` no SQL Server.
- Escrever script Flyway V1 em T-SQL com a estrutura base das tabelas.

Etapa 1.2 – Fundamentos do projeto Spring Boot

- Criar projeto Spring Boot com as dependências: Spring Web, Spring Data JPA, `mssql-jdbc`, Bean Validation, Flyway, Apache POI, JAXB, Lombok.
- Configurar `application.yml` apontando para o banco `ressarcimento` no SQL Server.
- Configurar Flyway, JPA/Hibernate com dialeto SQL Server.
- Configurar tratamento centralizado de exceções (`@ControllerAdvice`).
- Configurar logging estruturado.

Etapa 1.3 – Módulo Declarante

- Entidade JPA `Declarante`, repositório, serviço e controller REST.
- Endpoints: `POST /api/declarante`, `GET /api/declarante`.

- Validações Bean Validation conforme manual.
- Testes unitários do serviço.

Etapa 1.4 – Módulo Produtos (MATRI-NAC)

- Entidade JPA `ProdutoMatriz` e `ArquivoProdutos`.
- Serviço de importação de planilha (Apache POI) com relatório de erros por linha.
- Serviço de geração do XML `enviProdutoRessarcimento` com JAXB.
- Endpoints: `POST /api/produtos/importar`, `GET /api/produtos`, `POST /api/produtos/gerar-xml`, `GET /api/produtos/historico`.
- Testes unitários dos serviços de importação e geração de XML.
- **Validação manual:** gerar o XML e verificar conformidade com o leiaute do manual.

Etapa 1.5 – Módulo Pedidos (Versão 2.00)

- Entidades JPA `NotaSaida`, `ItemNotaSaida`, `NotaEntrada`, `ArquivoPedido`.
- Serviço de importação da planilha de operações com relatório de erros por linha.
- Lógica de agrupamento por `chaveNFe` de saída → `listaOperacoes` / `listaItens`.
- Lógica de deduplicação de chaves de entrada → `listaNFeEntrada`.
- Serviço de geração do XML `enviOperacaoRessarcimento` Versão "2.00" com JAXB.
- Endpoints: `POST /api/pedidos/importar`, `GET /api/pedidos`, `POST /api/pedidos/gerar-xml`, `GET /api/pedidos/historico`.
- Testes unitários dos serviços de importação, agrupamento e geração de XML.
- **Validação manual:** gerar o XML e verificar conformidade com o leiaute do manual.

Etapa 1.6 – Hardening do back-end

- Completar cobertura de testes unitários e de integração (Testcontainers com SQL Server, se viável).
- Revisar todas as validações de campos conforme o manual.
- Revisar logs, trilha de auditoria e tratamento de erros.
- Testar todos os endpoints via Postman/Insomnia com dados reais.
- Fazer upload dos XMLs gerados no DT-e para validação final pela SEFAZ.
- Documentar as APIs (Swagger/OpenAPI, se desejado).
- Apenas após aprovação completa desta etapa, avançar para a Fase 2.

FASE 2 — Frontend (iniciar somente após validação completa do back-end)

Etapa 2.1 – Decisão de tecnologia

- Com base no resultado da Fase 1, decidir entre Thymeleaf (server-side) ou SPA (React/Angular/Vue).
- Considerar o perfil dos usuários (fiscal/contábil): simplicidade, clareza de mensagens de erro e facilidade de uso.

Etapa 2.2 – Implementação da interface

- Tela de cadastro do declarante.
- Tela de upload da planilha de produtos, exibição de erros e download do XML.
- Tela de upload da planilha de operações, exibição de erros e download do XML.
- Tela de histórico de arquivos gerados (produtos e pedidos).
- Mensagens de erro claras, em português, adequadas ao perfil fiscal/contábil.

Etapa 2.3 – Testes de aceitação e entrega

- Testes com usuários reais (perfil fiscal/contábil).
- Ajustes de usabilidade.
- Empacotamento final e implantação.

5.8 Exemplos de código em Java

Forneça exemplos de código para as partes críticas:

- Configuração completa do `application.yml` para SQL Server com banco `ressarcimento`.
- Script Flyway `V1__criar_tabelas.sql` em T-SQL criando as tabelas principais (`NVARCHAR`, `DECIMAL`, `CHAR`, `DATETIME2`, `IDENTITY`).
- Entidade JPA `ProdutoMatriz` com anotações Bean Validation e mapeamento SQL Server.
- Entidade JPA `ItemNotaSaida` com relacionamentos.

- Serviço de importação de planilha de produtos com Apache POI (tratamento de erro por linha, retorno de relatório de erros).
- Serviço de geração do XML `enviProdutoRessarcimento` com JAXB.
- Serviço de geração do XML `enviOperacaoRessarcimento` Versão "2.00" com JAXB, incluindo:
    - Agrupamento por chave NF-e de saída.
    - Deduplicação de chaves de NF-e de entrada.
    - Omissão de tags opcionais quando não informadas.
- Controller REST para o módulo de produtos com tratamento de erros.
- Estrutura de testes unitários com JUnit 5 + Mockito para os serviços de geração de XML.

5.9 Extensões futuras

Comente brevemente as extensões previstas para o futuro:

- Suporte ao arquivo de pedidos **PMPF (Versão 1.00)** — projetar a arquitetura do back-end de forma a facilitar essa adição (ex.: padrão Strategy para geração de XML por tipo/versão de pedido).
- Suporte a múltiplas empresas (multi-tenant).
- Importação dos XMLs das NF-e/CT-e/MDF-e para preenchimento e validação automática das planilhas.
- Possível integração direta com o DT-e caso a SEFAZ/AM disponibilize API/webservice no futuro.
- Parametrização de versões de leiaute para facilitar atualização dos schemas XSD sem necessidade de alteração de código.

Organize a resposta nas seções: **Arquitetura**, **Modelo de Dados**, **Scripts Flyway T-SQL**, **Planilhas**, **APIs REST**, **Fluxo de Uso (back-end)**, **Validações**, **Roadmap (Fase 1 — Backend / Fase 2 — Frontend)**, **Exemplos de Código** e **Extensões Futuras**. Seja detalhado o suficiente para que uma equipe Java inicie o projeto com total segurança e boas práticas, priorizando a solidez do back-end antes de qualquer trabalho de interface.